

# DTO, Mapeamento e Swagger

O contrato da API separado do banco — e documentado pelo próprio código

*Conceito → DTO de entrada e de saída, records e mapper • Prática → springdoc-openapi + Swagger UI*

# O que vamos ver hoje

- Parte 1 — DTO: por que devolver a entidade não é um bom contrato, DTO de entrada x DTO de saída e records.
- Parte 2 — Mapeamento: o TarefaMapper, o Service como fronteira das entidades e as alternativas (MapStruct).
- Parte 3 — Swagger: o contrato OpenAPI gerado pelo springdoc, o Swagger UI e as anotações de documentação.
- Vamos sair hoje com a mesma API de /tarefas — com o mesmo JSON — só que agora com um contrato explícito e documentado.

## 1. DTO: o contrato da API

Request x Response, records, mass assignment

## 2. Mapeamento

toEntity, toResponse, updateEntity e MapStruct

## 3. Swagger e OpenAPI

springdoc, Swagger UI, @Operation, @Schema

# De onde viemos

- Na Aula 08, as tarefas foram para o PostgreSQL: a classe Tarefa virou @Entity e o Repository virou um JpaRepository.
- A entrada já tinha um DTO: o TarefaDTO, com as validações do @Valid.
- Mas a saída era a própria entidade: o Controller devolvia Tarefa e o Jackson transformava em JSON tudo o que tivesse getter.
- Hoje: cada lado ganha o seu DTO, um mapper faz a tradução, e o contrato passa a ser documentado pelo Swagger.

```
TarefaController.java (Aula 08)

// Aula 08 -- TarefaController
@PostMapping
public ResponseEntity<Tarefa> criar(
    @Valid @RequestBody TarefaDTO dto) {
    Tarefa criada = service.criar(dto);
    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(criada);    // <- a ENTIDADE
}

@GetMapping("/{id}")
public Tarefa buscarPorId(@PathVariable Long id) {
    return service.buscarPorId(id);
}
```

## PARTE 1

# DTO: o contrato da API

Separando o JSON que entra e sai das entidades do banco

# O problema: a entidade como contrato

- Devolvendo a entidade, o contrato da API passa a ser "tudo o que Tarefa tiver de getter" — ninguém decidiu isso explicitamente.
- O JSON fica amarrado à tabela: cada mudança na entidade é uma mudança no contrato, e os clientes quebram junto.
- Funciona hoje, com 8 campos simples. O problema aparece quando a entidade cresce: um campo interno, uma senha, um relacionamento.
- A saída é a mesma que a Aula 07 aplicou na entrada: um objeto só para carregar os dados através do HTTP.

## Entidade (@Entity)

Espelha a TABELA. Muda quando o banco muda. Tem relacionamentos, campos internos, anotações de JPA.

## DTO (Data Transfer Object)

Espelha o JSON. Muda só quando o CONTRATO muda. Sem regra de negócio, sem JPA: só dados.

# Quatro riscos de expor a entidade

| Risco           | O que acontece                                                                             | Exemplo no projeto                                                                       |
|-----------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Vazamento       | Um atributo novo na entidade aparece sozinho em toda resposta                              | Responsavel ganha "senha" ou "cpf" → sai em GET /responsaveis e em toda tarefa vinculada |
| Acoplamento     | Renomear um atributo ou uma coluna quebra os clientes                                      | dataPrazo vira prazo → a página index.html para de mostrar o prazo                       |
| Mass assignment | Se a entidade for usada na entrada, o cliente controla campos que só a API deveria decidir | POST com "id": 1 ou "concluida": true sobrescreve dados                                  |
| Loop e LAZY     | Relacionamento de mão dupla entra em loop no Jackson; LAZY estoura fora da sessão          | Por isso Responsavel não tem List<Tarefa> na Aula 08                                     |

Nenhum desses riscos é um erro de compilação: todos aparecem em produção, depois que alguém mudou a entidade achando que era só uma mudança interna.

# O que é um DTO

- DTO (Data Transfer Object) é um objeto feito só para carregar dados através de uma fronteira — aqui, a fronteira HTTP.
- Não tem regra de negócio nem anotação de JPA: tem os campos do JSON e, na entrada, as validações.
- O DTO É o contrato: o que está nele entra/sai; o que não está, não existe para o cliente.
- Uma entidade pode ter vários DTOs, um para cada uso: criar, atualizar, listar resumido, detalhar...

```
dto/TarefaResponseDTO.java

// o contrato da API, escrito num lugar só
public record TarefaResponseDTO(
    Long id,
    String titulo,
    boolean concluida,
    String responsavel,
    LocalDate dataPrazo,
    LocalDate dataCadastro,
    int prioridade,
    ResponsavelResponseDTO responsavelVinculado
) { }
```

# DTO de entrada x DTO de saída

|                | TarefaRequestDTO (entrada)                                            | TarefaResponseDTO (saída)                                               |
|----------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------|
| Quem monta     | O Jackson, a partir do JSON do corpo (@RequestBody)                   | O TarefaMapper, a partir da entidade                                    |
| Campos         | titulo, responsavel, dataPrazo, prioridade e, opcional, responsavelId | Os quatro primeiros + id, concluida, dataCadastro, responsavelVinculado |
| Validação      | @NotBlank, @NotNull, @FutureOrPresent, @Min/@Max                      | Nenhuma: quem gera é a própria API                                      |
| Protege contra | O cliente controlar id, concluida ou dataCadastro                     | Campos saindo sem ninguém decidir                                       |
| Usado em       | POST /tarefas e PUT /tarefas/{id}                                     | Toda resposta que devolve tarefa                                        |

Os campos em comum aparecem nos dois records. Não é duplicação: é o mesmo dado em dois contratos que podem evoluir separados. E o vínculo nem tem o mesmo formato nos dois lados (próximo slide).

# Records: o DTO do Java moderno

dto/TarefaDTO.java (Aula 08)

```
// Aula 08: classe (~100 linhas com os comentários)
public class TarefaDTO {

    @NotBlank(message = "Título é obrigatório")
    private String titulo;

    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    // ... mais 3 atributos, 3 getters, 3 setters
}
```

dto/TarefaRequestDTO.java (Aula 09)

```
// Aula 09: record
public record TarefaRequestDTO(

    @NotBlank(message = "Título é obrigatório")
    String titulo,

    @NotBlank(message = "Responsável é obrigatório")
    String responsavel,

    @NotNull @FutureOrPresent
    LocalDate dataPrazo,

    @NotNull @Min(1) @Max(5)
    Integer prioridade

) { }
```

*O compilador gera construtor, acessores (titulo(), sem "get"), equals, hashCode e toString. Sem setter: o DTO não muda no caminho entre o Controller e o Service.*

## A saída também tem o seu DTO

- O TarefaResponseDTO lista exatamente o que sai, na ordem em que sai.
- O responsável vinculado também é um DTO (ResponsavelResponseDTO): DTO de saída só aponta para DTO de saída.
- O JSON ficou IGUAL ao da Aula 08 — as páginas index.html e responsaveis.html nem perceberam a troca.
- @Schema não valida nada: só descreve o campo e dá o exemplo que o Swagger mostra (Parte 3).

```
dto/TarefaResponseDTO.java

@Schema(description = "Tarefa como a API devolve")
public record TarefaResponseDTO(

    @Schema(example = "1")
    Long id,

    @Schema(example = "Estudar DTO e Swagger")
    String titulo,

    // ... concluida, responsavel, dataPrazo,
    //      dataCadastro, prioridade

    // outro DTO, não a entidade Responsavel
    ResponsavelResponseDTO responsavelVinculado
) { }
```

## Entra um id, sai um objeto

- No cadastro, o vínculo com o responsável vai como "responsavelId": 1. O cliente só diz QUAL responsável; nome e e-mail já estão no banco.
- Na resposta, ele volta como "responsavelVinculado": {id, nome, email} — o ResponsavelResponseDTO.
- Quem transforma o id em entidade é o Service (findById, 404 se não existir). O mapper recebe o responsável pronto: mapper não consulta o banco.
- No PUT, sem responsavelId, a tarefa fica sem vínculo: PUT substitui tudo. Na tela, é o select  do formulário.

```
TarefaRequestDTO → TarefaResponseDTO

# requisição: POST /tarefas
{
  "titulo": "Revisar o contrato no Swagger",
  "responsavel": "Ana",
  "dataPrazo": "2026-12-01",
  "prioridade": 3,
  "responsavelId": 1
}

# resposta: 201 Created
{
  "id": 4, ...,
  "responsavelVinculado": {
    "id": 1,
    "nome": "Ana Souza",
    "email": "ana@satc.edu.br"
  }
}
```

# Mass assignment na prática

- Mass assignment: o cliente manda campos a mais no JSON, e a API copia tudo para o objeto que vai para o banco.
- Com o TarefaRequestDTO, id, concluida e dataCadastro nem existem na entrada: o Jackson ignora o que não tem onde guardar.
- Resultado: o id vem do banco, a tarefa nasce pendente e a data de cadastro é a de hoje — não importa o que o cliente mandou.
- Teste você mesmo no Swagger ou no Postman (a collection da aula já tem esse request).

o que o cliente manda x o que a API faz

```
# requisição
POST /tarefas
{
  "titulo": "Estudar DTO",
  "responsavel": "Ana",
  "dataPrazo": "2026-12-01",
  "prioridade": 4,
  "id": 999,
  "concluida": true,
  "dataCadastro": "2020-01-01"
}

# resposta: 201 Created
{ "id": 3, "titulo": "Estudar DTO",
  "concluida": false,
  "dataCadastro": "2026-10-06", ... }
```

# Responsável sem entidade no contrato

*Objetivo: aplicar no lado de responsáveis o mesmo modelo que o lado de tarefas já usa — sem mudar o JSON.*

- Transforme o ResponsavelDTO no record ResponsavelRequestDTO (mesmas validações).
- Crie o ResponsavelMapper (@Component): toEntity, toResponse (aceita null) e toResponseList.
- No TarefaMapper, apague o toResponsavelResponse privado e use o ResponsavelMapper, recebido pelo construtor.
- ResponsavelService e ResponsavelController passam a receber e devolver só DTOs.
- 1.1 — Se o JSON não mudou, o que se ganhou? (pense num atributo "senha")
- 1.2 — Por que o TarefaMapper deve usar o ResponsavelMapper?

## Pronto quando...

- No Swagger, a seção Schemas só tem nomes terminados em DTO
- O painel 📄 da responsaveis.html continua todo ✅
- Nenhum Controller importa nada de api.model

## PARTE 2

# Mapeamento entre DTO e entidade

Um lugar só para traduzir entre o contrato da API e o banco

# Onde a conversão acontece



Ida: ① o Jackson monta o DTO e o @Valid confere → ② o Service chama toEntity() → ③ o Repository faz o save() (INSERT/UPDATE)



Volta: ④ o Repository devolve a entidade → ⑤ o Service chama toResponse() → ⑥ o Jackson transforma o DTO em JSON



*A entidade vive do Service para baixo. O Controller recebe um DTO e devolve um DTO — ele nem importa api.model.*

# Quem conhece o quê

| Camada                 | Recebe                        | Devolve                            | Conhece a entidade? |
|------------------------|-------------------------------|------------------------------------|---------------------|
| TarefaController       | TarefaRequestDTO (do Jackson) | TarefaResponseDTO (para o Jackson) | Não                 |
| TarefaService          | TarefaRequestDTO ou ids       | TarefaResponseDTO                  | Sim — é a fronteira |
| TarefaMapper           | DTO ou entidade               | Entidade ou DTO                    | Sim — é a tradução  |
| TarefaRepository       | Tarefa                        | Tarefa / List<Tarefa>              | Só a entidade       |
| Hibernate + PostgreSQL | Tarefa                        | Linhas da tabela tarefas           | Só a entidade       |

*Por que converter no Service, e não no Controller? Com @Transactional no método, a conversão roda com a sessão do Hibernate aberta — e um relacionamento LAZY ainda pode ser carregado.*

# O TarefaMapper

- @Component: o Spring cria um TarefaMapper e injeta no TarefaService, igual faz com os repositórios.
- Um método por direção: toEntity (POST), updateEntity (PUT), toResponse (toda resposta) e toResponseList.
- O mapper não acessa o banco: o responsável chega pronto, buscado pelo Service a partir do responsavelId.
- Mapeamento manual: verboso, mas sem mágica — dá para ler, depurar com breakpoint, e o compilador reclama se um tipo mudar.
- Antes, essa tradução estava espalhada: new Tarefa(...) no criar(), setters repetidos no atualizar() e o Jackson lendo a entidade.

```
mapper/TarefaMapper.java

@Component
public class TarefaMapper {

    // o responsável já vem resolvido pelo Service
    public Tarefa toEntity(TarefaRequestDTO dto,
                          Responsavel responsavel) {
        Tarefa t = new Tarefa(null, dto.titulo(),
                              dto.responsavel(), dto.dataPrazo(),
                              dto.prioridade());
        t.setResponsavelVinculado(responsavel);
        return t;
    }

    public void updateEntity(Tarefa tarefa,
                            TarefaRequestDTO dto, Responsavel resp) {
        tarefa.setTitulo(dto.titulo());
        // ... responsavel, dataPrazo, prioridade, vínculo
    }

    public TarefaResponseDTO toResponse(Tarefa t) {
        return new TarefaResponseDTO(
            t.getId(), t.getTitulo(),
            t.isConcluida(), /* ... */);
    }
}
```

# O Service vira a fronteira das entidades

- Entra DTO, sai DTO: os métodos públicos do Service não devolvem mais Tarefa.
- Quem precisa da entidade (atualizar, concluir, vincular) usa a busca interna buscarEntidade(), que é private.
- buscarResponsavel() transforma o responsavelId em entidade: null = sem vínculo, inexistente = 404.
- O Controller não mudou quase nada: só os tipos dos parâmetros e dos retornos.
- O 404 continua vindo do mesmo lugar: `orElseThrow(() -> new TarefaNaoEncontradaException(id))`.

```

service/TarefaService.java

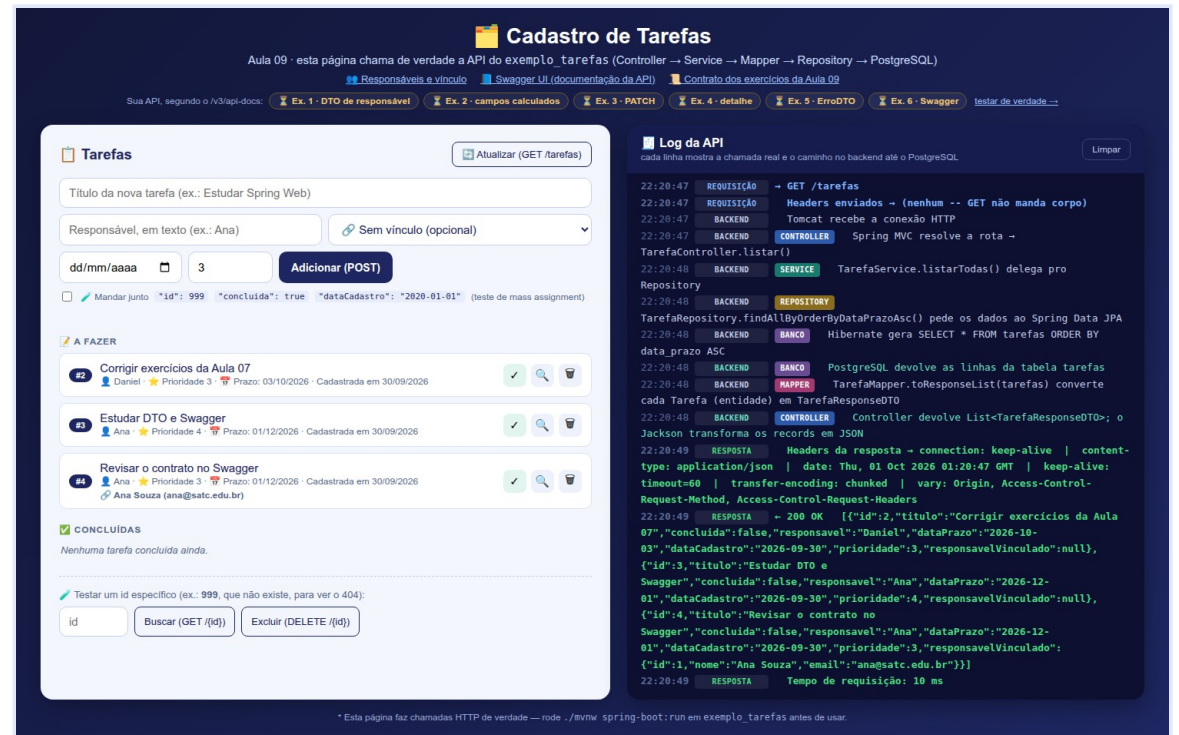
public TarefaResponseDTO criar(
    TarefaRequestDTO dto) {
    // o id vira entidade aqui (404 se não existir)
    Responsavel resp =
        buscarResponsavel(dto.responsavelId());
    Tarefa tarefa = mapper.toEntity(dto, resp);
    return mapper.toResponse(repository.save(tarefa));
}

public TarefaResponseDTO atualizar(
    Long id, TarefaRequestDTO dto) {
    Tarefa tarefa = buscarEntidade(id);
    mapper.updateEntity(tarefa, dto,
        buscarResponsavel(dto.responsavelId()));
    return mapper.toResponse(
        repository.save(tarefa));
}

// a entidade não sai deste Service
private Tarefa buscarEntidade(Long id) {
    return repository.findById(id)
        .orElseThrow(() ->
            new TarefaNaoEncontradaException(id));
}
    
```

# O contrato não mudou

- O JSON de GET /tarefas é idêntico ao da Aula 08: mesmos campos, mesma ordem, mesmos status.
- Mudou quem decide esse JSON: antes a entidade (sem querer), agora um DTO escrito para isso (de propósito).
- A única novidade é um campo OPCIONAL na entrada (responsavelId): quem não manda continua funcionando. Mudança compatível.
- A index.html ganhou o select  de vínculo, o teste de mass assignment, a camada MAPPER no log e a barra "Sua API".



The screenshot displays the 'Cadastro de Tarefas' application. The top navigation bar includes links for 'Responsáveis e vínculo', 'Swagger UI (documentação da API)', and 'Contrato dos exercícios da Aula 09'. Below the navigation bar, there are tabs for 'Ex. 1 - DTO de responsável', 'Ex. 2 - campos calculados', 'Ex. 3 - PATCH', 'Ex. 4 - detalhe', 'Ex. 5 - ErroDTO', and 'Ex. 6 - Swagger'. The main content area is divided into two sections: 'Tarefas' and 'Log da API'.

The 'Tarefas' section shows a form for adding a new task with fields for 'Título da nova tarefa (ex.: Estudar Spring Web)', 'Responsável, em texto (ex.: Ana)', and 'Sem vínculo (opcional)'. It also includes a date picker and a 'Adicionar (POST)' button. Below the form, there is a list of tasks under the heading 'A FAZER'. The tasks are: 'Corrigir exercícios da Aula 07' (Daniel, Prioridade 3, Prazo: 03/10/2026), 'Estudar DTO e Swagger' (Ana, Prioridade 4, Prazo: 01/12/2026), and 'Revisar o contrato no Swagger' (Ana, Prioridade 3, Prazo: 01/12/2026). Each task has a status icon (green checkmark, yellow warning, or red error) and a trash icon. At the bottom of the 'Tarefas' section, there is a 'CONCLUIDAS' section with a message 'Nenhuma tarefa concluída ainda.' and a 'Testar um id específico (ex.: 999, que não existe, para ver o 404)' button.

The 'Log da API' section shows a timeline of API requests and responses. The logs include the following information:

- 22:20:47: REQUISIÇÃO → GET /tarefas
- 22:20:47: REQUISIÇÃO Headers enviados → (nenhum -- GET não manda corpo)
- 22:20:47: BACKEND Tomcat recebe a conexão HTTP
- 22:20:47: BACKEND CONTROLLER Spring MVC resolve a rota → TarefaController.listar()
- 22:20:48: BACKEND SERVICE TarefaService.listarTodas() delega pro Repository
- 22:20:48: BACKEND REPOSITORY TarefaRepository.findAllByOrderByDataPrazoAsc() pede os dados ao Spring Data JPA
- 22:20:48: BACKEND BANCO Hibernate gera SELECT \* FROM tarefas ORDER BY data.prazo ASC
- 22:20:48: BACKEND BANCO PostgreSQL devolve as linhas da tabela tarefas
- 22:20:48: BACKEND MAPPER TarefaMapper.toResponseList(tarefas) converte cada Tarefa (entidade) em TarefaResponseDTO
- 22:20:48: BACKEND CONTROLLER Controller devolve List<TarefaResponseDTO>; o Jackson transforma os records em JSON
- 22:20:49: RESPOSTA Headers da resposta → connection: keep-alive | content-type: application/json | date: Thu, 01 Oct 2026 01:20:47 GMT | keep-alive: timeout=60 | transfer-encoding: chunked | vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
- 22:20:49: RESPOSTA → 200 OK [{"id":2,"titulo":"Corrigir exercícios da Aula 07","concluida":false,"responsavel":"Daniel","dataPrazo":"2026-10-03","dataCadastro":"2026-09-30","prioridade":3,"responsavelVinculado":null}, {"id":3,"titulo":"Estudar DTO e Swagger","concluida":false,"responsavel":"Ana","dataPrazo":"2026-12-01","dataCadastro":"2026-09-30","prioridade":4,"responsavelVinculado":null}, {"id":4,"titulo":"Revisar o contrato no Swagger","concluida":false,"responsavel":"Ana","dataPrazo":"2026-12-01","dataCadastro":"2026-09-30","prioridade":3,"responsavelVinculado":null}, {"id":1,"nome":"Ana Souza","email":"ana@satc.edu.br"}]
- 22:20:49: RESPOSTA Tempo de requisição: 10 ms

*http://localhost:8080/ — Controller → Service → Mapper → Repository → PostgreSQL, com a requisição real.*

# Manual, MapStruct ou ModelMapper?

| Abordagem   | Como funciona                                                                            | Vantagem                                             | Cuidado                                                   |
|-------------|------------------------------------------------------------------------------------------|------------------------------------------------------|-----------------------------------------------------------|
| Manual      | Uma classe @Component com um método por direção, campo a campo                           | Sem mágica: dá para ler e depurar                    | Verboso; campo novo exige lembrar do mapper               |
| MapStruct   | Você escreve a interface; um processador de anotações gera a implementação na compilação | Rápido e seguro: erro de mapeamento aparece no build | Configuração no pom.xml; campos calculados pedem @Mapping |
| ModelMapper | Copia por reflexão, casando os nomes dos campos em tempo de execução                     | Quase nenhum código                                  | Erros só aparecem rodando; mais lento                     |

Na disciplina seguimos com o mapeamento manual. O MapStruct é o próximo passo natural — e o código que ele gera é o mesmo que escrevemos à mão (próximo slide).

# MapStruct gera o código que escreveríamos

```

ResponsavelMapper.java

// você escreve só a interface
@Mapper(componentModel = "spring")
public interface ResponsavelMapper {

    ResponsavelResponseDTO toResponse(
        Responsavel responsavel);

    Responsavel toEntity(
        ResponsavelRequestDTO dto);
}

// pom.xml: dependência org.mapstruct:mapstruct
// + mapstruct-processor no maven-compiler-plugin
    
```

```

ResponsavelMapperImpl.java (gerado)

// gerado no build (target/generated-sources)
@Component
public class ResponsavelMapperImpl
    implements ResponsavelMapper {

    public ResponsavelResponseDTO toResponse(
        Responsavel responsavel) {
        if ( responsavel == null ) {
            return null;
        }
        Long id = responsavel.getId();
        String nome = responsavel.getNome();
        String email = responsavel.getEmail();
        return new ResponsavelResponseDTO(
            id, nome, email );
    }
}
    
```

*Testado com MapStruct 1.6.3 + Java 25 no projeto da aula: ele reconhece records e o construtor da entidade.*

# Um DTO para cada uso

## TarefaRequestDTO

Criar e substituir (POST, PUT). Obrigatórios, menos o responsavelId.

→ já no projeto

## TarefaResponseDTO

Toda resposta de tarefa. Pode ter campos calculados.

→ já no projeto

## TarefaPatchDTO

Atualização parcial (PATCH). Todos os campos opcionais.

→ exercícios

## TarefaResumoDTO

Tarefa enxuta dentro de outro DTO: id, titulo, concluida, prazo.

→ exercícios

## ResponsavelDetalheDTO

DTO composto: o responsável com as tarefas dele — sem loop.

→ exercícios

## ErroDTO

Até o erro é uma resposta: um formato só para todo 400 e 404.

→ exercícios

*Não existe "o DTO da tarefa": cada contrato tem as suas regras. Mesmos campos com regras diferentes = DTOs diferentes.*

## Campos calculados no DTO de saída

*Objetivo: mostrar quantos dias faltam e se a tarefa está atrasada — sem coluna nova e sem migration.*

- Acrescente long diasRestantes e boolean atrasada ao TarefaResponseDTO, depois de prioridade.
- Calcule os dois no TarefaMapper.toResponse(): ChronoUnit.DAYS.between(hoje, prazo); atrasada = pendente e prazo antes de hoje.
- Documente os dois campos com @Schema(description, example).
- 2.1 — Por que não viram colunas? O que daria errado gravando "atrasada" no cadastro?
- 2.2 — Por que calcular no backend, e não em cada frontend?

### Pronto quando...

- Prazo daqui a 7 dias → diasRestantes = 7
- Prazo ontem e pendente → atrasada = true
- A index.html mostra "⚠ Atrasada" com borda vermelha

# Atualização parcial com PATCH

*Objetivo: corrigir só o título de uma tarefa atrasada, sem reenviar o prazo (o problema do 3.2 da Aula 08).*

- Crie o record TarefaPatchDTO: os cinco campos (inclusive responsavelId), todos opcionais.
- Título e responsável: @Pattern("(?s).\*\\S.\*"); prazo: @FutureOrPresent; prioridade: @Min(1) @Max(5).
- TarefaMapper.applyPatch(tarefa, dto, responsavel): copia só o que for diferente de null.
- TarefaService.atualizarParcial() (usa o buscarResponsavel) e @PatchMapping("/{id}").
- 3.1 — Por que não reaproveitar o TarefaRequestDTO?
- 3.2 — O que o PATCH com "null = não mexer" não consegue fazer?

## Pronto quando...

- {"titulo": "x"} → 200, só o título muda
- {} → 200, nada muda
- {"responsavelId": 2} → troca o vínculo
- {"prioridade": 9} → 400
- A edição da index.html passa a usar PATCH

## Detalhe do responsável, com as tarefas dele

*Objetivo: devolver o responsável com as tarefas dentro — o que a entidade não podia fazer na Aula 08 por causa do loop.*

- Crie `TarefaResumoDTO(id, titulo, concluida, dataPrazo)` e `ResponsavelDetalheDTO(..., totalTarefas, tarefasPendentes, tarefas)`.
- `TarefaMapper.toResumo()` e `ResponsavelMapper.toDetalhe(responsavel, resumos)`.
- `ResponsavelService.buscarDetalhe(id)` e `GET /responsaveis/{id}` (404 se não existir).
- ⚠ O `ResponsavelMapper` NÃO injeta o `TarefaMapper`: seria uma dependência circular e a aplicação não sobe.
- 4.1 — Por que o DTO não cai no loop do Jackson?
- 4.2 — Por que `TarefaResumoDTO`, e não o `TarefaResponseDTO` completo?

### Pronto quando...

- `GET /responsaveis/1` traz as tarefas vinculadas
- `totalTarefas` e `tarefasPendentes` batem com a lista
- Responsável sem tarefas → `"tarefas": []`

## PARTE 3

# Swagger e OpenAPI

Documentação gerada a partir do próprio código — e testável no navegador

# OpenAPI, Swagger e springdoc

- Na Aula 06 vimos que uma API precisa de um contrato. Até aqui, o contrato estava em tabelas no README e no EXERCICIOS.md.
- Documentação escrita à mão envelhece: o código muda e o texto fica para trás.
- Com o springdoc, o contrato é gerado a partir do código que está rodando: rota nova, DTO novo e validação nova aparecem sozinhos.
- As anotações servem só para o que o código não tem como dizer: resumos, exemplos e status possíveis.

## OpenAPI

A especificação: formato padrão (JSON/YAML) que descreve rotas, parâmetros, corpos e respostas.

## Swagger

A família de ferramentas em volta da especificação. A mais usada: o Swagger UI.

## springdoc-openapi

Lê os Controllers e DTOs do Spring, gera o OpenAPI e serve o Swagger UI.

# Uma dependência e o Swagger no ar

- springdoc-openapi-starter-webmvc-ui: não faz parte do Spring, então a <version> é obrigatória.
- Linha 2.x = Spring Boot 3 (a 2.9.1 foi compilada contra o Boot 3.5.16, o do projeto). A 3.x é para o Spring Boot 4.
- Sem nenhuma propriedade, já funciona. As do projeto só deixam o padrão explícito e ajustam a página.
- OpenApiConfig (opcional): um @Bean OpenAPI com título, versão, descrição e contato.

```
pom.xml + application.properties

<!-- pom.xml -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>
    springdoc-openapi-starter-webmvc-ui
  </artifactId>
  <version>2.9.1</version>
</dependency>

# application.properties
springdoc.api-docs.path=/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
springdoc.default-produces-media-type=
  application/json
springdoc.swagger-ui.try-it-out-enabled=true
```

# Swagger UI: <http://localhost:8080/swagger-ui.html>

## API de Tarefas

Aula 09 OAS 3.1

</v3/api-docs>

API REST de tarefas e responsáveis da disciplina de Backend (SATC 2026.1). Todo corpo de requisição e de resposta é um **DTO** (veja a seção *Schemas*, lá embaixo) e todo erro 400/404 sai no formato **ErroDTO**.

Contact Prof. Daniel Plácido

Servers

<http://localhost:8080> - Generated server url

### Responsáveis

Cadastro de responsáveis e consulta das tarefas de cada um

- GET **/responsaveis** Lista os responsáveis
- GET **/responsaveis/{id}** Detalha um responsável
- GET **/responsaveis/{id}/tarefas** Lista as tarefas de um responsável

- Topo: o bloco "info" do OpenApiConfig.
- Um grupo por @Tag, cada endpoint com o resumo do @Operation.
- Clique numa rota → Try it out → Execute: a requisição é real, igual ao Postman.
- Lá embaixo, Schemas: todo DTO que entra ou sai da API.

# Try it out: o corpo já vem preenchido

**POST**
**/tarefas**
Cria uma tarefa

id, concluida e dataCadastro são definidos pela API, não pelo cliente. Com responsavelId (opcional)

**Parameters**

No parameters

**Request body** required

Edit Value | Schema

```

{
  "titulo": "Estudar DTO e Swagger",
  "responsavel": "Ana",
  "dataPrazo": "2026-12-01",
  "prioridade": 3,
  "responsavelId": null
}

```

- O corpo de exemplo vem do @Schema(example = ...) de cada componente do TarefaRequestDTO.
- Sem example, o Swagger preencheria "string", 0 e "2026-01-01".
- "required" e os limites (1 a 5) vêm das anotações de validação: @NotBlank, @Min, @Max.
- Os status documentados (201, 400) vêm dos @ApiResponse.

# Anotações do Swagger

| Anotação     | Onde        | Para quê                                           | Exemplo                                         |
|--------------|-------------|----------------------------------------------------|-------------------------------------------------|
| @Tag         | Controller  | Nome e descrição do grupo                          | @Tag(name = "Tarefas", description = "...")     |
| @Operation   | Método      | Resumo e descrição do endpoint                     | @Operation(summary = "Cria uma tarefa")         |
| @ApiResponse | Método      | Cada status possível e o formato do corpo          | @ApiResponse(responseCode = "201", ...)         |
| @Parameter   | Parâmetro   | Descrição e exemplo de @PathVariable/@RequestParam | @Parameter(example = "1") @PathVariable Long id |
| @Schema      | DTO / campo | Descrição e example (o valor do Try it out)        | @Schema(example = "Ana") String responsavel     |

Nenhuma delas muda o comportamento da API: tire todas e ela responde igual. Elas só completam o documento OpenAPI.

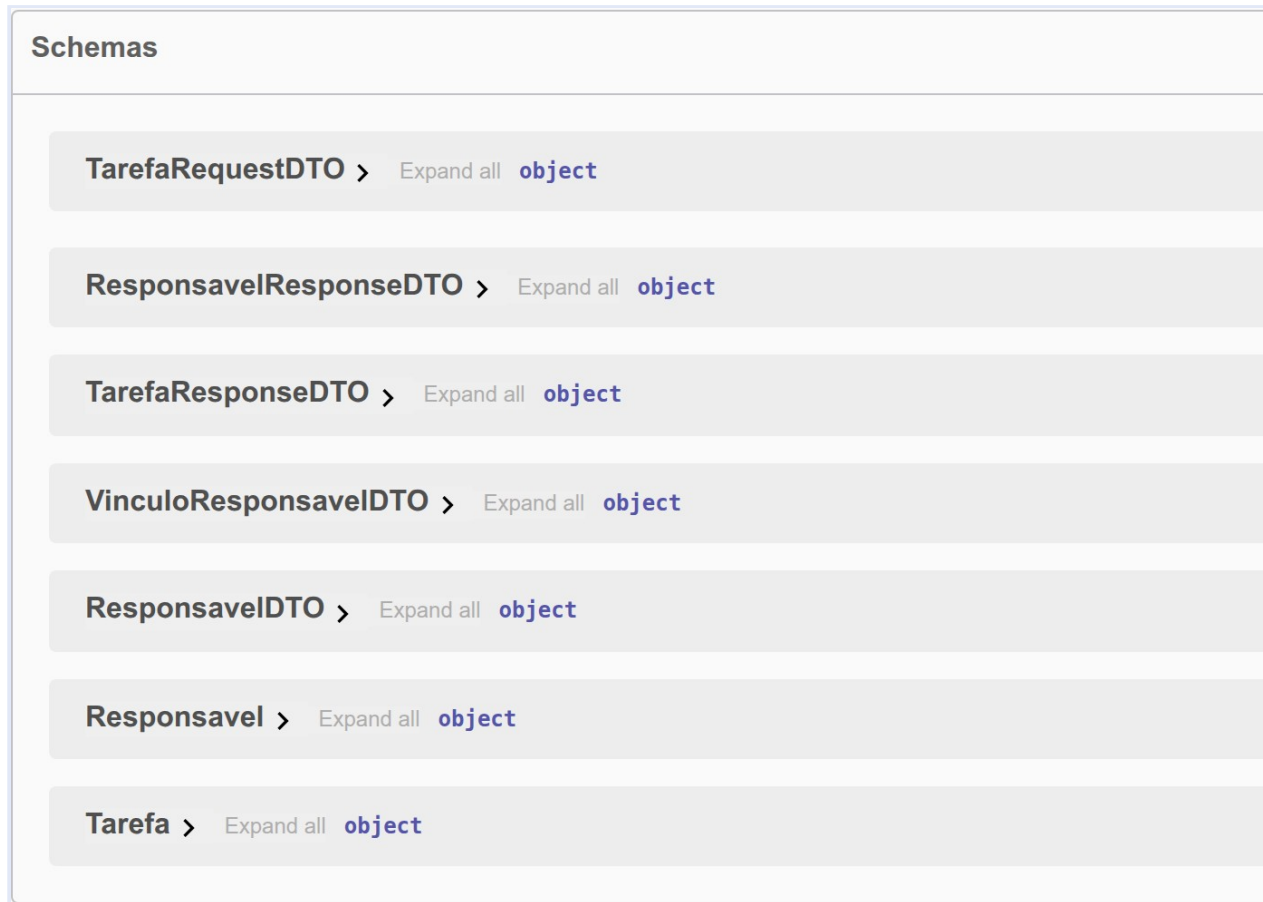
# O springdoc lê a assinatura, não o corpo

- Da assinatura `ResponseEntity<TarefaResponseDTO>`, o springdoc tira o FORMATO da resposta.
- O STATUS só é decidido lá dentro, em `ResponseEntity.status(CREATED)` — e o springdoc não executa nem lê o corpo do método.
- Sem `@ApiResponse`, o POST aparece como 200. É o que acontece hoje com o POST `/responsaveis` (Exercício 6).
- Os erros também não aparecem na assinatura: o 404 nasce de uma exceção lançada lá no Service.

```
controller/TarefaController.java

@Operation(summary = "Cria uma tarefa")
@ApiResponse(responseCode = "201",
             description = "Tarefa criada")
@ApiResponse(responseCode = "400",
             description = "Corpo inválido",
             content = @Content)
@PostMapping
public ResponseEntity<TarefaResponseDTO> criar(
    @Valid @RequestBody TarefaRequestDTO dto) {
    TarefaResponseDTO criada = service.criar(dto);
    return ResponseEntity
        .status(HttpStatus.CREATED) // 201
        .body(criada);
}
```

## Schemas: o vazamento aparece na tela



- A seção Schemas lista tudo o que entra e sai da API.
- No projeto da aula ainda aparecem Responsavel e Tarefa: o ResponsavelController devolve as entidades.
- É o banco vazando para o contrato — e agora dá para ver.
- O Exercício 1 tira as duas de lá.

### Critério do Exercício 1

Na seção Schemas só devem existir DTOs — um bom hábito para qualquer API.

## /v3/api-docs: o contrato para máquinas

- O Swagger UI é só um dos programas que leem o documento OpenAPI — o feito para pessoas.
- Postman: Import → Link → <http://localhost:8080/v3/api-docs> e a collection aparece pronta.
- Geradores de código (openapi-generator) criam o cliente em TypeScript, Kotlin... sem redigitar os DTOs.
- Testes de contrato, API gateways, comparação entre versões para achar mudanças que quebram clientes.
- A página contrato.html da aula lê esse JSON para conferir os Exercícios 1 e 6.

```
GET /v3/api-docs

{
  "openapi": "3.1.0",
  "info": { "title": "API de Tarefas",
            "version": "Aula 09" },
  "paths": {
    "/tarefas": {
      "post": {
        "tags": ["Tarefas"],
        "summary": "Cria uma tarefa",
        "requestBody": { "content": {
          "application/json": { "schema": {
            "$ref": "#/components/schemas/TarefaRequestDTO" }}}},
        "responses": { "201": ..., "400": ... }
      } } },
    "components": { "schemas": { ... } }
  }
}
```

# Erro padronizado com ErroDTO

*Objetivo: um formato só para todo 400 e 404 — no lugar do texto puro e do JSON padrão do Spring.*

- Crie ErroDTO(status, erro, mensagem, caminho, timestamp, campos) e CampoErroDTO(campo, mensagem).
- ApiExceptionHandler: um handler para os dois "não encontrado" e um para MethodArgumentNotValidException.
- @ApiResponse de 400/404 apontando para @Schema(implementation = ErroDTO.class).
- Não crie um @ExceptionHandler(Exception.class) "pega-tudo".
- 5.1 — Por que dois formatos de erro são um problema?
- 5.2 — Por que nunca devolver o stack trace?

## Pronto quando...

- GET /tarefas/999 → 404 em ErroDTO, "campos": []
- POST /tarefas {} → 400 com 4 itens em "campos"
- As páginas mostram "(ErroDTO)" no log

# ResponsavelController documentado

*Objetivo: deixar o grupo de responsáveis tão documentado quanto o de tarefas — e com os status certos.*

- @Tag(name = "Responsáveis", description = ...) no Controller.
- @Operation(summary) e @ApiResponse em todo endpoint, inclusive o GET /responsaveis/{id}.
- @Parameter(example = "1") nos path variables; @Schema(example) em cada componente dos DTOs novos.
- Importe <http://localhost:8080/v3/api-docs> no Postman.
- 6.1 — De onde veio o 200 do POST /responsaveis?
- 6.2 — OpenAPI e Swagger são a mesma coisa? Para que serve o /v3/api-docs?

## Pronto quando...

- Grupo "Responsáveis" com descrição e resumos
- POST documenta 201 e 400 (e nenhum 200)
- Try it out já vem com "Ana Souza", não "string"

## Rodando o exemplo\_tarefas da Aula 09

- Banco na porta 5435 e name: aula09-tarefas no docker-compose.yml: sem o name, o Docker usaria o nome da pasta — o mesmo da Aula 08 — e as duas aulas dividiriam o volume.
- Nenhuma migration nova: DTO e mapper não mudam o banco.
- Swagger: /swagger-ui.html · Cadastro: / · Responsáveis: /responsaveis.html · Contrato: /contrato.html
- Na primeira execução, o Maven Wrapper baixa o springdoc: é preciso internet.

```
terminal

cd "Aula 09/exemplo_tarefas"

docker compose up -d      # PostgreSQL, porta 5435
docker compose ps         # "healthy"

./mvnw spring-boot:run    # Windows: .\mvnw.cmd

# no navegador
http://localhost:8080/swagger-ui.html
http://localhost:8080/contrato.html

# o contrato em JSON
curl http://localhost:8080/v3/api-docs
```

# contrato.html: confira cada exercício

✓ Checklist dos exercícios

✓ 4 ⌚ 21 ✗ 0

Rodar o contrato

✓ segue o contrato · ⌚ ainda não implementado · ✗ implementado, mas fora do contrato · — não testado.  
O teste cria uma tarefa temporária ([contrato] ...), usa e apaga no final. Se não existir nenhum responsável, cadastra um chamado Contrato Aula 09 (não existe DELETE de responsável).

base

Já vem pronto no projeto

✓ GET /v3/api-docs devolve o documento OpenAPI  
OpenAPI 3.1.0, 8 rotas

✓ POST /tarefas com "responsavelId" → a tarefa já nasce vinculada  
entrou "responsavelId": 1, saiu "responsavelVinculado": {id, nome, email}

Ex. 1

Responsável sem entidade no contrato

⌚ Nenhuma entidade (Tarefa, Responsavel) na seção Schemas  
ainda aparecem as entidades: Responsavel, Tarefa

✓ GET /responsaveis continua devolvendo [{id, nome, email}]  
1 responsavel(is)

✓ GET /responsaveis/{id}/tarefas no mesmo formato do GET /tarefas/{id}  
campos: concluida, dataCadastro, dataPrazo, id, prioridade, responsavel, responsavelVinculado, titulo

Ex. 2

Campos calculados no DTO de saída

⌚ Toda tarefa traz diasRestantes (número) e atrasada (true/false)  
o TarefaResponseDTO ainda não tem esses campos

⌚ diasRestantes e atrasada batem com dataPrazo e concluida  
o TarefaResponseDTO ainda não tem esses campos

⌚ Prazo hoje → diasRestantes = 0 e atrasada = false  
o TarefaResponseDTO ainda não tem esses campos

- A página roda 25 testes contra a SUA API: cria uma tarefa temporária, testa e apaga no final.
- ✓ segue o contrato · ⌚ ainda não implementado · ✗ implementado, mas fora do contrato.
- Ela não conhece o seu código Java: só olha as respostas HTTP e o /v3/api-docs — como qualquer cliente da API.
- No projeto recém-baixado: 4 ✓ e 21 ⌚. Meta da entrega: 24 ✓ (25 com o item opcional do Exercício 5).
- As outras páginas também mudam sozinhas: a barra "Sua API" lê o /v3/api-docs e a edição passa a usar PATCH quando ele existe.
- É a página que o professor abre para corrigir.

# Boas práticas de DTO e documentação

| Prática                                        | Por quê                                                           |
|------------------------------------------------|-------------------------------------------------------------------|
| Nunca devolver a entidade JPA                  | O contrato não pode mudar sozinho quando o banco muda             |
| DTO de entrada separado do de saída            | Na entrada, o cliente não controla id, datas nem estado           |
| Um DTO por uso (criar, patch, resumo, erro...) | Regras diferentes pedem contratos diferentes                      |
| Conversão só no mapper                         | Uma regra, um lugar: sem new Entidade(...) espalhado pelo projeto |
| Documentar todos os status (@ApiResponse)      | O springdoc não descobre o 201 nem os erros sozinho               |
| example em todo @Schema                        | O Try it out vira um teste pronto, não um formulário com "string" |
| Erro também é DTO, sem stack trace             | Formato estável para o cliente, nada de detalhe interno exposto   |

## RECAPITULANDO

# O que levamos para casa

- O DTO é o contrato: a entidade espelha a tabela, o DTO espelha o JSON — e os dois mudam por motivos diferentes.
- Entrada e saída têm DTOs separados; records deixam isso curto e imutável.
- O mapper é o único lugar que traduz entre DTO e entidade; a entidade vive do Service para baixo.
- O springdoc gera o contrato OpenAPI a partir do código; as anotações completam o que ele não adivinha.
- Com Swagger, o contrato fica navegável, testável (Try it out) e legível por máquina (/v3/api-docs).

### Dever de casa

Os 6 exercícios do EXERCICIOS.md (12 subexercícios), com a contrato.html em 24 .

Entrega: código no GitHub + respostas no corpo do e-mail, assunto "DTO - <seu nome>".

## Para ir além

- springdoc-openapi — documentação e FAQ: [springdoc.org](https://springdoc.org)
- Especificação OpenAPI: [spec.openapis.org/oas/latest.html](https://spec.openapis.org/oas/latest.html)
- Swagger UI e demais ferramentas: [swagger.io/tools/swagger-ui](https://swagger.io/tools/swagger-ui)
- Records em Java (JEP 395): [openjdk.org/jeps/395](https://openjdk.org/jeps/395)
- MapStruct — guia de referência: [mapstruct.org/documentation/stable/reference/html](https://mapstruct.org/documentation/stable/reference/html)
- PATCH (RFC 5789) e JSON Merge Patch (RFC 7396): [rfc-editor.org](https://rfc-editor.org)
- OWASP — Mass Assignment Cheat Sheet: [cheatsheetseries.owasp.org](https://cheatsheetseries.owasp.org)
- Repositório da disciplina: [github.com/placidodaniel/backend-aulas-satc](https://github.com/placidodaniel/backend-aulas-satc)